

课程笔记

AI Agent Loop 到底是不是神技？Greg Isenberg 和 Ras Mic 聊清楚了

Codex

2026 年 6 月 22 日



视频作者/频道: KrillinAI 小林

发布日期: 2026-06-11

视频时长: 22:32

视频链接: <https://www.bilibili.com/video/BV1D2Ev6YEkn/>

目录

1	Agent Loop 的承诺：从人类在环到代理自循环	2
2	全自动 Loop 的失控点：假设、Token 与产品愿景	5
3	什么时候可以放手：实验、原型与有限目标	8
4	可落地的闭环：Cursor、GitHub 与 Greptile 代码审查	10
5	边界条件：二元反馈、人类反馈与创造性工作	12
6	总结与延伸：最好的 Loop 是有约束的人机协作	16

1 Agent Loop 的承诺：从人类在环到代理自循环

1.1 为什么所有人都在谈论 Agent loop

这段开场要解决一个很基本的问题：Agent loop（代理闭环）到底是什么，为什么它会让 AI 行业里的很多人兴奋，又为什么 Ras Mic 一开始就给出强烈警告。视频里的语气很直接：这个概念真实存在，也确实有实际用法；同时，把它理解成“让代理一直自动做下去”会给普通建造者带来很高的成本和质量风险。

所谓“承诺”，来自一个很诱人的想象：人类只写一次目标、PRD 或 Markdown 规格说明，代理随后自己生成结果、检查结果、修复结果，再继续下一步。对开发者、创业者和产品团队来说，这听起来像把日常提示、检查、返工和沟通压缩成一次启动动作。它把“我持续提示 AI”升级成“我设计一个能自己跑的系统”。

这个承诺之所以有吸引力，是因为它抓住了一个真实痛点。普通的人类在环工作流里，人需要不断检查中间结果：着陆页是否符合定位，认证流程是否能用，后端数据模型是否撑得住，用户看到页面后是否愿意注册。每一步都要看、试、改。Agent loop 试图把这些反馈动作的一部分交给系统自己处理。

这个差异在白板图里很直观：人类在环时，人的提示、测试和批准仍然留在流程中间；代理自循环时，反馈回路更多发生在 AI agent 与结果之间。

1.2 human-in-the-loop：普通建造者熟悉的控制结构

Ras Mic 先画出的第一种模型，是普通建造者已经在使用的 human-in-the-loop（人类在环）。人类给 Cursor、Claude、Codex 或其他 AI 代理下指令，代理生成输出，人类查看、测试、批准或要求修改。这里的“环”并没有消失，区别在于反馈来自人。

可以把一个待办事项应用作为例子。创始人先让代理生成着陆页，因为产品需要对外发布和收集等待名单；着陆页能表达价值主张后，再让代理做认证；认证流程可用后，再推进后端、任务模型、数据库、权限和部署。每个阶段都需要人类确认，因为产品的目标、用户感受和商业节奏会在过程中变清楚。

human-in-the-loop（人类在环）

human-in-the-loop 指人类审查、测试、批准和产品判断被放在工作流内部。它像一个产品经理、测试者和预算负责人同时坐在流程里：产品经理负责方向，测试者负责发现问题，预算负责人负责决定是否继续消耗时间和 Token。

这个控制面让代理可以快速生成和修复，也让人类能在方向偏离时及时接管。对普通建造者来说，人类在环常常是质量控制、成本控制和产品判断的同一个入口。

人类在环的成本很明显：慢，需要人看中间结果，需要人做判断。它的价值也同样明显：错误会比较早暴露，预算消耗可以被及时叫停，产品愿景不会完全交给模型猜测。视频后面所有关于全自动 loop 的批评，都以这个控制结构作为参照。

1.3 代理自循环：从一次触发到连续执行

第二种模型才是视频讨论的主角：Agent loop。人类触发一次，代理开始生成结果；系统对结果进行评估，或者接收某种反馈；这个反馈进入下一轮上下文，代理继续行动。人类参与被压缩到启

¹ 视频画面时间区间：00:02:31–00:02:41。

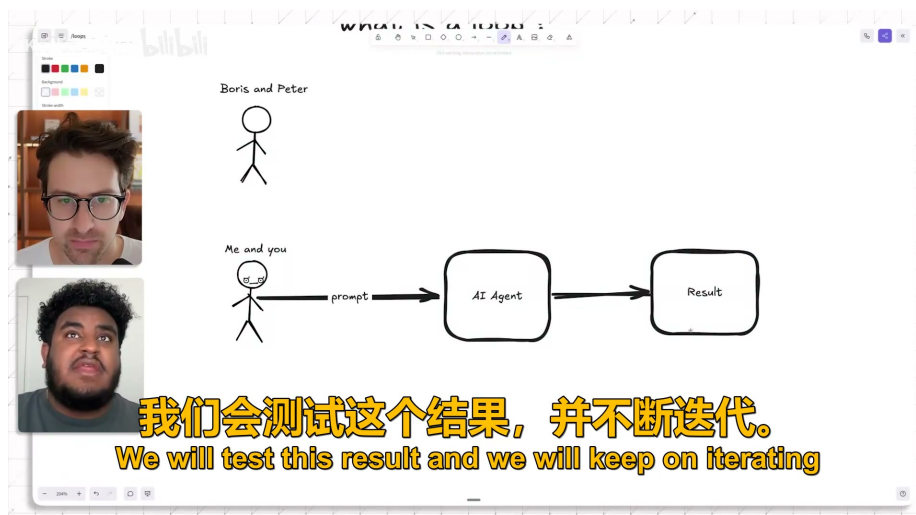


图 1: human-in-the-loop 模式：人类检查结果，并把判断带回下一轮。¹

动、偶尔检查或最终验收，循环内部的大量判断交给代理与反馈机制。

Agent loop（代理闭环）的工作定义

Agent loop（代理闭环）是一个最小人类 steering 的自动化工作流：代理先生成输出，再评估输出或接收反馈，然后把这个结果反馈进下一次行动。循环持续运行，直到达到目标、触发停止条件、耗尽预算，或由人类接管。

它的核心组件是：代理行动、反馈信号、下一步行动、停止规则。缺少反馈信号，循环会变成自动猜测；缺少停止规则，循环会持续消耗 Token 和注意力。

这里的关键词是“反馈”。如果代理只是连续生成代码，那只是批量输出；如果代理能读测试结果、审查意见、评分、错误日志或人工评论，并把这些信号变成下一步修改，它才形成真正的闭环。一个闭环的质量，取决于反馈信号是否可靠、范围是否足够小、系统是否知道何时停下。

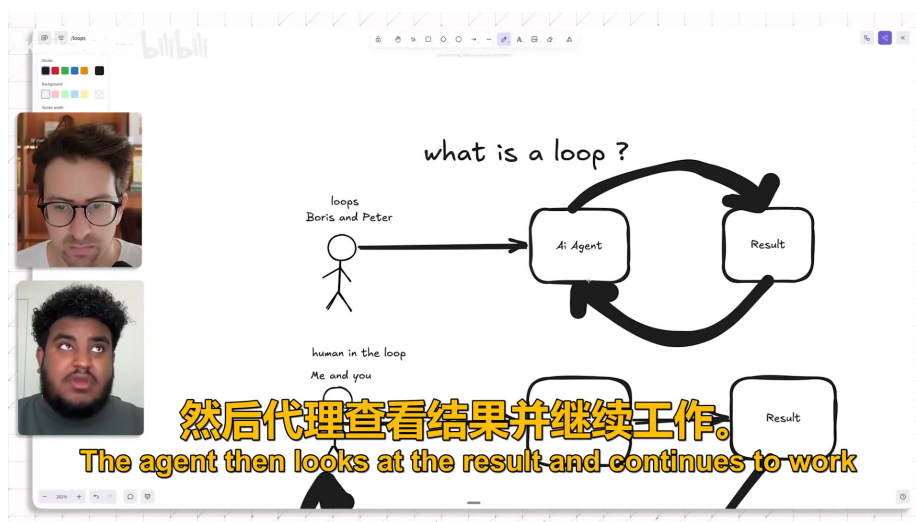


图 2: Agent loop 模式：结果回流到代理，代理继续执行下一轮。²

²视频画面时间区间：00:03:56–00:04:07。

1.4 为什么 PRD 或 Markdown 规格说明听起来像解决方案

全自动 loop 的想象通常会绑定到 PRD 或 Markdown spec。人类把产品目标、功能需求、页面结构、设计要求和技術约束写进一个文档，然后交给代理执行。表面上看，这像是在给代理提供“所有必要信息”。如果文档足够完整，代理似乎就能从着陆页一路做到认证、后端、部署和修复。

这种想法有一个合理起点：规格说明确实能减少沟通成本。对重复任务、模板任务和实验任务来说，写清楚输入、输出、验收标准和限制条件，会显著提高 AI 代理的可用性。真正的难点藏在“所有必要信息”这几个字里。产品工作里的必要信息往往会在构建过程中出现，初始文档很难一次性覆盖。

Ras Mic 的待办事项应用例子正好说明了这一点。一个页面是否“对”，取决于用户是谁、他们为什么注册、等待名单承诺是否可信、视觉风格是否符合市场、认证流程是否影响转化、后端模型是否服务未来功能。很多判断只有在看到中间版本后才会浮现。PRD 能描述目标，它很难穷尽所有产品取舍。

1.5 承诺背后的真实吸引力

Agent loop 的吸引力可以拆成三层。

- **速度吸引力**：人类减少逐轮提示，代理自己完成更多连续步骤。
- **系统吸引力**：工作从“一个提示得到一个答案”变成“一个机制持续改进输出”。
- **规模吸引力**：只要反馈信号可靠，同一套循环可以重复处理许多相似任务。

这三层吸引力都是真实的。代码审查、测试驱动修复、固定模板内容生成、批量页面生产，都可能从闭环中获益。后续章节的 Greptile 代码审查例子会展示一种更可靠的闭环：Cursor 写代码，GitHub 承载 pull request，Greptile 给出审查和评分，Cursor 根据反馈继续修复。

第 1 章需要先把概念边界立住。Agent loop 的价值来自“反馈能否闭合”，靠代理长时间运行无法保证结果变好。反馈质量低时，代理会把错误假设自动放大；反馈质量高时，代理才可能把局部任务推向可验收状态。

1.6 第一层警告：人类从环里退出后，控制面也会减少

Ras Mic 在开场阶段已经埋下了后续批评：当人类只参与一次启动，代理就必须在大量细节上自行做决定。界面长什么样、功能如何排序、数据模型如何设计、错误状态如何处理、用户路径如何组织，这些都可能被代理用默认模式填充。

这会带来一种很隐蔽的错觉：系统看起来在快速前进，实际可能在快速固化未经批准的选择。对普通建造者来说，速度本身没有意义；有意义的是朝着正确产品方向的速度。人类在环的作用，正是让方向检查发生在还来得及修改的时候。

因此，本章的结论很朴素：Agent loop 是一种反馈系统，魔法词这个理解会误导使用者。它能把代理、输出和反馈连接起来，也会把规格说明里的空白放大成实际产品决策。下一章要处理的，就是这些空白如何变成失控点。

1.7 本章小结

- Agent loop（代理闭环）指代理生成输出、接收或评估反馈，再把反馈带入下一步行动的自动化 workflow。

- human-in-the-loop（人类在环）把人类审查、测试、批准、预算控制和产品判断放在循环内部。
- 代理闭环的承诺来自速度、系统化和规模化；它的风险来自人类控制面减少后，代理必须替人做大量隐性产品选择。
- PRD 或 Markdown 规格说明可以降低沟通成本，却很难一次性表达全部产品语境、UX 取舍和架构细节。
- 判断一个闭环是否可靠，要先问反馈信号是否清楚、范围是否有限、停止规则是否存在。

2 全自动 Loop 的失控点：假设、Token 与产品愿景

2.1 聪明开发者类比：失控从规格说明的空白开始

第 2 章把 Agent loop 的风险讲得更具体。Ras Mic 使用了一个创业团队类比：Greg 和 Ras Mic 正在创业，他们雇了一位很聪明的开发者，给出应用目标和功能清单，然后让这位开发者一路把完整产品做出来。开发者很聪明，执行力很强，最终仍然会在大量细节上自行判断。

这些判断包括产品外观、用户体验、信息架构、技术架构、错误状态、数据模型、页面层级、默认值和发布路径。任何真实产品都会遇到这种问题：初始 brief 写不完全部细节，团队成员之间也很难把想法完全对齐。开发者只能补齐空白；代理也会补齐空白。

这个类比的锋利之处在于，它把“AI 代理失控”从神秘能力问题拉回到普通产品开发问题。代理没有读心能力。人类给出的 PRD 或 Markdown spec 越含糊，代理需要自行填补的面积越大；填补面积越大，偏离产品愿景的概率越高。

不完整规格说明会制造隐性假设

把一个含糊产品 brief 交给聪明开发者，然后要求对方独立交付完整产品，结果很可能是一套看似完成、实际充满未经创始人批准决策的系统。Agent loop 也一样：PRD 或 Markdown 规格说明里的空白，会被代理用默认假设填满。

这些假设可能落在 UI、UX、架构、状态管理、权限、文案、数据结构和发布策略上。问题到最后才暴露时，团队面对的已经是一堆互相依赖的实现选择，修复成本远高于一两个局部小错误。

这个 warningbox 是本章的核心。所谓失控，很多时候表现为代理按自己的推断顺利完成了大量工作，而非系统崩溃。顺利完成与符合产品愿景之间存在距离，距离越晚被发现，返工越贵。

2.2 PRD 与 Markdown spec 的边界

PRD 的价值在于压缩沟通：目标是什么，功能有哪些，用户是谁，验收标准是什么，技术约束有哪些。Markdown spec 的价值类似，它让代理可以读取结构化要求和上下文。对于小功能、明确脚本、固定页面模板和测试驱动修复，这类文档非常有用。

边界也同样明确。产品愿景包含大量难以提前写死的东西：一个按钮的语气是否合适，首屏承诺是否可信，登录流程是否过早，空状态是否减轻用户挫败感，架构是否服务下个月的商业计划。很多取舍需要看到中间产物后才会出现。文档可以传递目标，无法替代持续判断。

视频里提到服务行业的经验：即便人类团队努力提取客户脑子里的所有想法，也经常会遇到“这里漏了”“我原本想要的是另一种形式”。这说明问题来自表达本身的有限性。AI 代理可以读更多上下文，也仍然会受制于初始表达里的空白。

2.3 隐性假设如何变成产品债务

“假设”听起来像一个很轻的词，实际会变成产品债务。它像代码里的默认参数：当默认值只影响一个局部函数时，改起来很容易；当默认值进入数据库、路由、权限和用户路径后，后续每一步都会围绕它继续生长。

在全自动 loop 里，假设至少会沿着三条线传播。

- **体验线**：代理选择页面布局、交互路径和文案语气，用户感受被默认决策塑造。
- **工程线**：代理选择状态管理、数据模型、API 结构和错误处理，后续功能会依赖这些选择。
- **商业线**：代理决定哪些功能先做、哪些信息突出、哪些用户动作被鼓励，产品定位被实现细节固化。

这些债务的麻烦之处在于，它们常常一开始看起来很合理。直到创始人看到完整产品，或者用户试用后产生困惑，团队才意识到很多选择没有经过讨论。人类在环的意义，就是把这种发现提前到着陆页、认证、后端、核心流程这些阶段性节点。

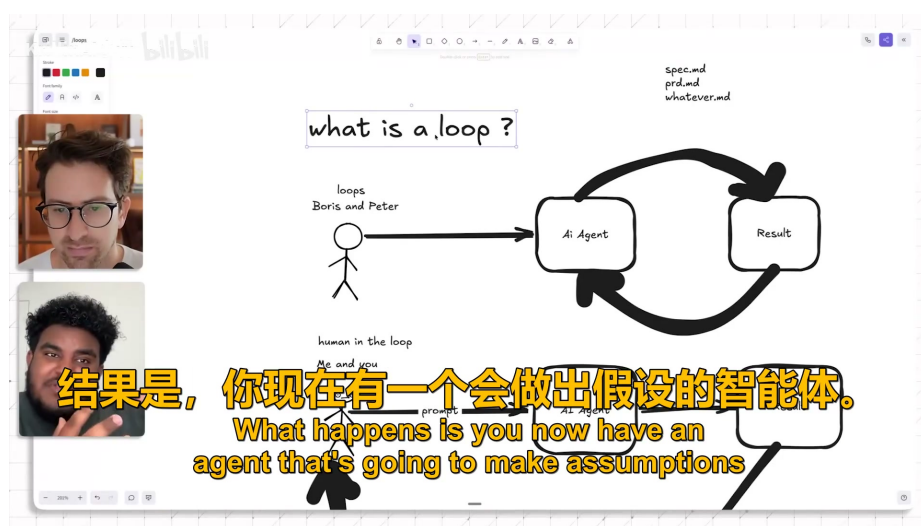


图 3: 规格说明留下空白时，代理会在循环中自动填入假设。³

2.4 Token 预算：循环每跑一轮都会收费

Ras Mic 对 Token 成本的提醒非常 blunt。Token 在这里可以理解为模型使用量和边际成本的代理变量：提示、上下文、代码、反馈、日志、修复建议都会消耗 Token。一个开放式 loop 越会自我迭代，消耗越容易从“几次调用”变成“长时间后台燃烧”。

这种 Token 风险在画面中被讲得很直白：循环听起来很酷，持续运行时也会持续烧钱。

这个预算约束可以用一个短式子表达：

$$T \leq B$$

- T ：某个 Agent loop 在一次任务里消耗的 Token 总量，也代表模型调用成本和机会成本的粗略代理变量。

³ 视频画面时间区间：00:06:25–00:06:42。

⁴ 视频画面时间区间：00:11:32–00:11:40。

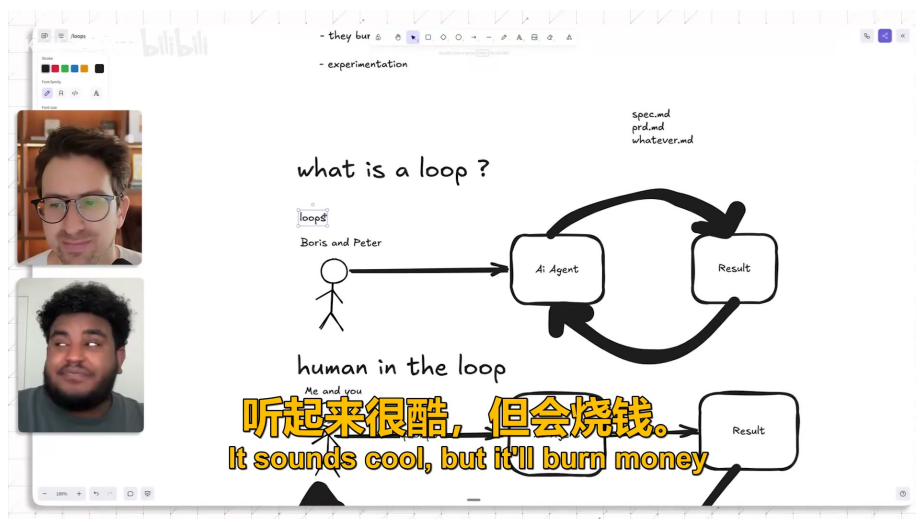


图 4: 开放式循环会持续消耗 Token；预算约束必须进入设计。⁴

- B: 团队愿意为该任务投入的 Token 预算，包括金钱预算、等待时间和注意力预算。

这条式子没有复杂数学含义，它只是在提醒普通建造者：循环的边际成本必须小于预算。高预算团队可以把长时间自我修复当作研发投入；普通建造者需要在循环启动前设置范围、上限和人工检查点。

视频里把 Boris 和 Peter 描述为拥有更大 Token 空间的探索者，这应按 speaker claim 处理。这个说法的教学价值，不在于验证某个人究竟花了多少钱，而在于说明资源环境不同会改变工程策略。研究团队可以用大量预算探索未来能力，普通创业者必须对当下产品和账单负责。

2.5 /goal 与 /loop: 命令名不同，风险结构相同

Greg 追问 /goal 与 loop 的关系，Ras Mic 的回答很清楚：这些命令在不同工具里名字可能不同，例如 /goal、/loop 或其他 slash command；高层机制都是让用户给出目标、附上文档，然后让代理持续执行直到完成。

这类命令的吸引力很强，因为它把复杂工作包装成一个简单入口。用户输入目标，附上 PRD 或 Markdown spec，系统开始行动。界面越顺滑，越容易让人忽略背后的关键问题：反馈信号来自哪里，停止条件是什么，谁检查中间结果，谁承担错误成本。

因此，/goal 和 /loop 更适合被理解成“自动化入口”，替代产品判断这个定位会制造风险。入口越强大，边界越需要清楚。把一个模糊产品愿景交给自动化入口，系统会把模糊性转化成实现细节；把一个明确、可测、范围有限的任务交给入口，系统才有机会稳定地产出价值。

2.6 产品愿景无法完全写进一次性文档

视频中有一个很实在的产品例子：趋势会变化，视觉偏好会变化，团队今天觉得某种“液态玻璃”风格很好，明天可能又想调整比例、质感或呈现方式。这类判断很难在初始文档里写到足够准确，因为团队自己的想法也在中途变化。

这属于产品探索的正常状态，文档质量只是其中一个变量。产品愿景由用户反馈、创始人品味、市场变化、竞争态势、技术约束和团队资源共同塑造。一次性 PRD 只能捕捉某个时刻的理解，无法承载构建过程中出现的新信息。